How the project uses D-Wave

Code walk-through, modes of use, checks, estimates, budget guard, and the 1-second test

`Projects_master_HPQC/scripts/QuantumComputing/Dimod`

Abstract

This document explains, file by file and line by line, how the SK-game campaign talks to D-Wave:

- which Ocean functions it uses, what they can do, and how the code harnesses them;
- the six ways ("modes") of running an annealer in this project;
- what `qpu --check` actually does, and how the QPU time is estimated (three levels, with the formulas);
- how the budget guard (`--budget-seconds`) makes sure a run never exceeds a given QPU time;
- how the results are turned into success probabilities, time-to-solution and the Zephyr/Pegasus comparison.

The code excerpts are read directly from the source files when the document is compiled, so the line numbers are those of the version in the repository. Section 13 gives the exact commands for a test that uses at most one second of QPU time, followed by a plan to spend the 180 s of this month before 4 November.

Contents

1 Where the D-Wave code lives **2**

2 The Ocean SDK: what each package can do, and what we use it for **2**
2.1 `dwave-cloud-client`: connection, solvers, configuration, timing model . . . 2
2.2 `dwave-system`: samplers for the QPU, composites, mock . . . 3
2.3 `dimod`: the problem and the results . . . 4
2.4 `minorminer` and `dwave-graphs`: embeddings and hardware graphs . . . 4
2.5 `dwave.embedding` (inside `dwave-system`): chains in and out . . . 5
2.6 `dwave-samplers` and `dwave-preprocessing` . . . 6

3 The modes of use **6**

4 Life of one QPU call, line by line **7**

5 What `qpu --check` does (and why it costs nothing) **9**

6 How QPU time is estimated: three levels **11**
6.1 Level 1: the rough plan (`main_simulation.py plan`) . . . 11
6.2 Level 2: the solver's timing model (`--check`, budget guard) . . . 11
6.3 Level 3: the charge (`qpu_access_time`) . . . 11

7 The budget guard: `qpu --budget-seconds S` **12**

8 Broken chains, read by read **13**

9 Ground states **15**

10 Ising or QUBO? And is the problem trivial? **16**

11 From reads to results **17**

12 How the D-Wave path was tested **18**

13 The one-second test, and a plan for the 180 s **19**
13.1 Test: at most 1 s of QPU time, both topologies 19
13.2 Spending the 180 s before 4 November 19

1 Where the D-Wave code lives

All paths are relative to `Dimod/src/unlearnable_games_explanation/`.

file	role
`skgame/optim/problem.py`	the SK problem W; `to_bqm()` builds the dimod model
`skgame/optim/embedding.py`	topologies (Zephyr, Pegasus), embedding strategies (clique, heuristic), chain statistics, embedding cache
`skgame/optim/backends.py`	`AnnealBackend`: one object per `device:topology:strategy`; builds the sampler, embeds, samples, collects statistics, estimates time
`skgame/optim/quantum.py`	`QuantumAnnealingSolver`: the older single-solver path (`DWaveCliqueSampler`/`EmbeddingComposite`), used by the planner benchmark
`skgame/optim/solvers.py`	`DimodSamplerSolver` (generic dimod wrapper) and the classical dimod samplers
`main_simulation.py`	the campaign: stages `init`, `instances`, `classical`, `embed`, `qpu`/`sim`, `analyze`; the QPU budget guard; the analysis
`tests/test_embedding.py`, `tests/test_campaign.py`	tests of all the above, including the QPU path against D-Wave's mock QPU

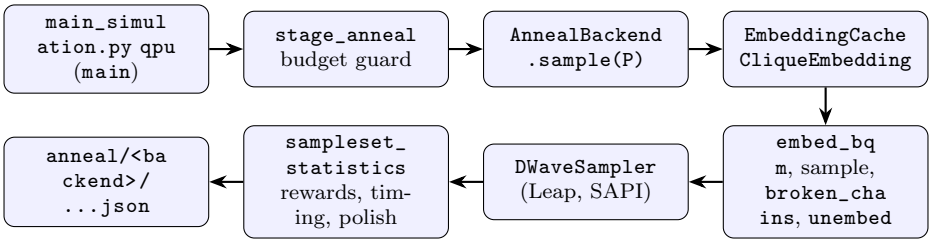


Figure 1: The path of one QPU call through the code.

2 The Ocean SDK: what each package can do, and what we use it for

D-Wave's Python SDK (*Ocean*) is a set of packages, all installed by `cineca/requirements.txt`. For each package: what it offers, then exactly how this project uses it.

2.1 `dwave-cloud-client`: connection, solvers, configuration, timing model

Capabilities.

- it talks to the Leap *Solver API* (SAPI) over HTTPS;
- it reads the token from the configuration file or from the environment (`DWAVE_API_TOKEN`);
- it selects solvers with *feature filters* such as `solver={"topology__type": "zephyr"}`;
- it exposes every solver's `properties`, including the timing data;
- it estimates the cost of a problem (`estimate_qpu_access_time`);
- it submits problems asynchronously;
- it provides the `dwave` command line: `config`, `auth`, `leap`, `solvers`, `ping`, `setup`.

How we use it.

- Through `DWaveSampler` (next subsection), which wraps a cloud-client `Solver`.

- `solver.name` identifies the QPU in the logs and keys the embedding cache, e.g. `Advantage2_system1`.
- `solver.estimate_qpu_access_time` is the second level of our time estimate (§6).
- On the command line: `dwave config create`, `dwave setup --oob`, `dwave solvers -la`, `dwave ping` (`Dimod/LEAP_TOKEN.md`).

2.2 `dwave-system`: samplers for the QPU, composites, mock

Capabilities.

- `DWaveSampler`: a dimod sampler bound to one structured QPU. It exposes `nodelist`/`edgelist` (the working qubits and couplers), `properties`, `parameters` (the accepted keyword arguments) and `to_networkx_graph()`. Its `sample()` accepts `num_reads`, `annealing_time`, `anneal_schedule`, `auto_scale`, `answer_mode`, `h_gain_schedule`, `anneal_offsets`, `programming_thermalization`, `readout_thermalization`, ...
- Composites that turn a structured sampler into one that accepts any problem:
  - `EmbeddingComposite` finds a heuristic embedding on every call;
  - `FixedEmbeddingComposite` uses a given embedding;
  - `LazyFixedEmbeddingComposite` finds one embedding and reuses it;
  - `DWaveCliqueSampler` uses cached all-to-all clique embeddings, scales with the extended J range and computes chain strengths.
- `dwave.system.testing.MockDWaveSampler`: an offline imitation of a QPU of any topology and size, which returns realistic timing fields.

How we use it.

- `DWaveSampler(solver={"topology__type": ...})` picks a Zephyr or Pegasus QPU (code below).
- `to_networkx_graph()` gives the *real* working graph. The embedding is computed on it, so it avoids dead qubits.
- Every QPU call does explicitly what `FixedEmbeddingComposite` does: `embed_bqm`, then `DWaveSampler.sample`, then `unembed_sampleset`, with our cached clique embedding. Doing it by hand keeps the *physical* reads, from which the broken chains of every read are counted (§8). The code was checked to give read-for-read the same results as the composite. The Zephyr and Pegasus back-ends go through identical code and differ only in hardware and embedding.
- `DWaveCliqueSampler` and `EmbeddingComposite` remain available in `QuantumAnnealingSolver` (§3).
- `MockDWaveSampler` is used in the tests and in the dry runs.

`skgame/optim/backends.py`, lines 84–114: building the child sampler (QPU, embedded simulator or logical simulator) and reading its hardware graph

```
84      # ------------------------------------------------------------- hardware graph ----
85      @property
86      def child(self):
87          """The structured sampler the embedding maps onto."""
88          if self._child is None:
89              if self.device == "qpu":
90                  from dwave.system import DWaveSampler
91                  s = self.settings
92                  self._child = DWaveSampler(solver=s.solver_name or self.topology.solver_filter)
93              elif self.embedded:
```

```
                import dimod
                from dwave.samplers import SimulatedAnnealingSampler
                G = self._target = self.topology.ideal_graph()
                self._child = dimod.StructureComposite(SimulatedAnnealingSampler(), list(G.nodes), list(G.edges)
    )
            else:
                from dwave.samplers import SimulatedAnnealingSampler
                self._child = SimulatedAnnealingSampler()
        return self._child

    @property
    def target(self):
        """networkx graph of the working qubits and couplers."""
        if self._target is None and self.embedded:
            child = self.child                # building a sim child also sets the ideal graph
            if self._target is not None:
                pass
            elif hasattr(child, "to_networkx_graph"):
                self._target = child.to_networkx_graph()   # real working graph, keeps the topology metadata
            else:
                raise ValueError(f"{self.spec}: cannot read the hardware graph of {type(child).__name__}")
        return self._target
```

2.3 `dimod`: the problem and the results

Capabilities.

- `BinaryQuadraticModel` in SPIN or BINARY form;
- `SampleSet`, which holds the reads: `record.sample`, `record.energy`, `record.num_occurrences`, extra vectors such as `chain_break_fraction`, plus `info` (timing, embedding context). It also offers `aggregate()`, `first` and `resolve()`;
- a common `Sampler` interface (`sample(bqm, **kwargs)`, `parameters`);
- helper composites such as `StructureComposite`, and reference samplers such as `ExactSolver`.

How we use it.

- The SK problem becomes an Ising BQM whose energy is exactly $H(S) = -\sum_{i<j} W_{ij} S_i S_j$.
- `StructureComposite` restricts simulated annealing to the Zephyr or Pegasus graph (the `sim:*:clique` back-ends).
- `ExactSolver` checks the statistics code in the tests.

`skgame/optim/problem.py`, lines 45–50: the SK problem as a dimod BQM ($h = 0$, $J_{ij} = -W_{ij}$)

```
    def to_bqm(self):
        """dimod BinaryQuadraticModel (SPIN) with E = H(S). Lazy import."""
        import dimod
        iu = np.triu_indices(self.N, 1)
        quad = {(int(i), int(j)): -float(self.W[i, j]) for i, j in zip(*iu)}
        return dimod.BinaryQuadraticModel({}, quad, 0.0, dimod.SPIN)
```

2.4 `minorminer` and `dwave-graphs`: embeddings and hardware graphs

Capabilities.

- `minorminer.busclique.find_clique_embedding` returns all-to-all embeddings made of line segments, optimised for chain length and avoiding defects, with an on-disk cache;
- `busgraph_cache(G).largest_clique()` gives the largest K_N the graph supports;
- `minorminer.find_embedding` is the general heuristic;

- `dwave.graphs` (formerly `dwave_networkx`) provides `zephyr_graph`, `pegasus_graph`, coordinate converters and drawing functions.

How we use it.

- The clique strategy and its maximum size (below), which give the `largest_problem` printed by `qpu --check`.
- The heuristic baseline.
- The ideal graphs for the simulator and for the figures in `visualization/embedding_explanation`.

`skgame/optim/embedding.py`, lines 101–115: the all-to-all strategy: embedding and largest clique

```
101 class CliqueEmbedding(EmbeddingStrategy):
102     """Native all-to-all embedding (minorminer.busclique): every chain has the same, minimal-ish
103     length (Zephyr: about N/16 + 2 qubits; Pegasus: about N/12 + 2), deterministic and fast.
104     This is what DWaveCliqueSampler uses internally."""
105
106     name = "clique"
107
108     def embed(self, N, target):
109         from minorminer.busclique import find_clique_embedding
110         emb = find_clique_embedding(N, target, use_cache=True)
111         return {int(k): [int(q) for q in v] for k, v in emb.items()}
112
113     def max_size(self, target):
114         from minorminer.busclique import busgraph_cache
115         return len(busgraph_cache(target).largest_clique())
```

`skgame/optim/embedding.py`, lines 157–164: chain statistics; `n_qubits` feeds the readout-time model

```
157 def chain_stats(embedding: dict) -> dict:
158     """Chain lengths of an embedding: the main predictor of chain breaks and of the energy
159     compression imposed by the chain couplings."""
160     if not embedding:
161         return {"n_qubits": 0, "chain_len_mean": float("nan"), "chain_len_max": 0, "chain_len_min": 0}
162     L = np.array([len(c) for c in embedding.values()])
163     return {"n_qubits": int(L.sum()), "chain_len_mean": float(L.mean()),
164             "chain_len_max": int(L.max()), "chain_len_min": int(L.min())}
```

2.5 `dwave.embedding` (inside `dwave-system`): chains in and out

Capabilities.

- `embed_bqm` maps a logical BQM onto qubits: it splits each J_{ij} over the couplers between two chains and adds $-c$ on the chain couplers;
- `unembed_sampleset` maps reads back to logical spins;
- `chain_strength.uniform_torque_compensation` gives $c = p\, J_{\mathrm{rms}} \sqrt{\bar{d}}$;
- `chain_breaks`: `majority_vote`, `weighted_random`, `discard`, `MinimizeEnergy`.

How we use it.

- The chain strength is `partial(uniform_torque_compensation, prefactor=p)`, with $p =$ `chain_strength_prefactor`.
- The chain-break method is chosen by name.
- `MinimizeEnergy` has to be wrapped in a named function, because `unembed_sampleset` reads `__name__`. Testing revealed this.

skgame/optim/backends.py, lines 147–152: the keyword arguments for the physical sampler

```
147     def sample_kwargs(self) -> dict:
148         """Keyword arguments for the (physical) sampler itself."""
149         s = self.settings
150         if self.device == "qpu":
151             return {"num_reads": s.num_reads, "annealing_time": s.annealing_time}
152         return {"num_reads": s.sim_num_reads, "num_sweeps": s.sim_num_sweeps}
```

skgame/optim/backends.py, lines 211–221: chain-break resolution by name

```
211 def _chain_break_method(name: str, bqm, emb):
212     from dwave.embedding import chain_breaks as cb
213     if name == "minimize_energy":
214         resolver = cb.MinimizeEnergy(bqm, emb)
215 
216         def minimize_energy(samples, chains):    # unembed_sampleset needs a __name__
217             return resolver(samples, chains)
218         return minimize_energy
219     if name in ("majority_vote", "weighted_random", "discard"):
220         return getattr(cb, name)
221     raise ValueError(f"chain_break_method must be one of {CHAIN_BREAK_METHODS}")
```

2.6 `dwave-samplers` and `dwave-preprocessing`

Capabilities.

- classical samplers with the dimod interface: `SimulatedAnnealingSampler`, `TabuSampler`, `SteepestDescentSolver`, `TreeDecompositionSolver`, `RandomSampler`;
- `SpinReversalTransformComposite` (gauge transforms) and other pre-processing.

How we use it.

- SA, tabu and steepest descent are classical baselines (`solvers.py`).
- SA is also the engine of the `sim:logical` and `sim:*:clique` back-ends.
- Spin-reversal transforms are discussed as a mitigation (`explanation_project.pdf`, noise section) but are not yet wired in.

3 The modes of use

mode	what runs	when to use it	QPU time / token
`sim:logical`	SA on K_N directly (no hardware graph)	reference for the annealing pipeline	none / no
`sim:zephyr:clique`, `sim:pegasus:clique`	the *embedded* problem, chains included, solved by SA on the ideal graph (`StructureComposite`)	test the whole embedding path; classical view of chain effects	none / no
mock QPU (tests)	`MockDWaveSampler` in place of `DWaveSampler`, same code path as the QPU	tests, dry runs	none / no
`qpu:zephyr:clique`, `qpu:pegasus:clique`	real QPU: `embed_bqm` + `DWaveSampler` + `unembed_sampleset` with our clique embedding	the campaign	yes / yes
`qpu:*:minorminer`	real QPU with a heuristic embedding	baseline only	yes / yes

mode	what runs	when to use it	QPU time / token
`QuantumAnnealingSolver`	`DWaveCliqueSampler` (clique, extended J range) or `EmbeddingComposite` (heuristic each call)	the planner benchmark (`skgame/optim/benchmark.py`)	yes / yes

`skgame/optim/quantum.py`, lines 49–58: the single-solver path: `DWaveCliqueSampler` or `EmbeddingComposite`

```
49      def make_sampler(self):
50          if self.use_simulator:
51              from dwave.samplers import SimulatedAnnealingSampler
52              return SimulatedAnnealingSampler()
53          if self.clique:
54              from dwave.system import DWaveCliqueSampler
55              return DWaveCliqueSampler(solver=self.solver_name) if self.solver_name else DWaveCliqueSampler()
56          from dwave.system import DWaveSampler, EmbeddingComposite
57          base = DWaveSampler(solver=self.solver_name) if self.solver_name else DWaveSampler()
58          return EmbeddingComposite(base)
```

The campaign embeds explicitly (the steps of `FixedEmbeddingComposite`) rather than using `DWaveCliqueSampler`, so that both topologies run *identical* code: same embedding strategy, same chain-strength rule, same scaling. Their results then differ only because of the hardware.

4 Life of one QPU call, line by line

`skgame/optim/backends.py`, lines 154–199: `AnnealBackend.sample`: embed, sample, keep the physical reads, unembed

```
154     def sample(self, P: PlannerProblem, rng=None) -> dict:
155         """One call. For embedded back-ends this does explicitly what FixedEmbeddingComposite does
156         (embed -> sample -> unembed), so that the PHYSICAL reads are available in between: from
157         them `broken_chains` tells, read by read, which logical spins had a broken chain."""
158         rng = np.random.default_rng(rng)
159         bqm = P.to_bqm()
160         kw = self.sample_kwargs()
161         params = getattr(self.child, "parameters", {}) or {}
162         if self.device == "sim" and "seed" in params:
163             kw["seed"] = int(rng.integers(2**31))
164         broken, phys_counts, cs_used, emb = None, None, None, None
165         t0 = time.perf_counter()
166         if self.embedded:
167             import dimod
168             from dwave.embedding import EmbeddedStructure, broken_chains, unembed_sampleset
169             from dwave.embedding.chain_strength import uniform_torque_compensation
170             emb = self.embedding(P.N)
171             structure = EmbeddedStructure(self.child.edgelist, emb)
172             target = structure.embed_bqm(bqm, chain_strength=partial(
173                 uniform_torque_compensation, prefactor=self.settings.chain_strength_prefactor),
174                 smear_vartype=dimod.SPIN)
175             cs_used = structure.chain_strength
176             phys = self.child.sample(target, **kw)
177             phys.resolve()
178             chains = [emb[i] for i in range(P.N)]
179             broken = np.asarray(broken_chains(phys, chains), bool)          # (physical rows, N)
180             phys_counts = np.asarray(phys.record.num_occurrences, int)
181             ss = unembed_sampleset(phys, structure, bqm,
182                                    chain_break_method=_chain_break_method(self.settings.chain_break_method, bqm,
    emb),
183                                    chain_break_fraction=True)
184             ss.info.update({k: v for k, v in phys.info.items() if k not in ss.info})
185         else:
186             ss = self.child.sample(bqm, **kw)
187             ss.resolve()
188         wall = time.perf_counter() - t0
189         out = sampleset_statistics(ss, P, wall, polish=self.settings.polish, broken=broken, phys_counts=
    phys_counts,
190                                    discard=self.settings.chain_break_method == "discard")
191         info = out["info"]
192         info.update(backend=self.spec, solver=self.solver_id(), settings=asdict(self.settings),
```

```
                    sample_kwargs={k: v for k, v in kw.items() if k != "seed"})
        if emb is not None:
            info["embedding"] = chain_stats(emb)
            info["chain_lengths"] = [len(emb[i]) for i in range(P.N)]
            info["chain_strength"] = float(cs_used) if np.isscalar(cs_used) else None
            info["max_abs_logical_coupling"] = float(np.max(np.abs(P.W)))
        return out
```

What happens, in order:

1. `P.to_bqm()`: the SK problem as an Ising BQM.
2. `self.embedding(P.N)`: the clique embedding of K_N on the solver's real graph, read from `embeddings/<solver>/clique/N....json` or computed and cached.
3. `EmbeddedStructure(child.edgelist, emb).embed_bqm(bqm, chain_strength=...)` computes c with the chain-strength callable, splits each J_{ij} over the couplers between two chains and puts $-c$ on the chain couplers.
4. `DWaveSampler.sample(target, num_reads, annealing_time)` submits the physical problem to SAPI. `auto_scale` is on by default and divides by the largest coefficient.
5. `broken_chains(phys, chains)`: for every physical read and every logical spin, whether its chain is broken (qubits disagreeing).
6. `unembed_sampleset(phys, structure, bqm, chain_break_method=..., chain_break_fraction=True)` maps the reads back to logical spins.
7. `ss.resolve()` waits for the asynchronous answer.
8. `sampleset_statistics` recomputes every reward from the spins: it is exact whatever scaling was applied. It aggregates identical reads, records `info["timing"]` (in μs) and the read-averaged chain-break fraction, and optionally polishes every distinct read with greedy descent.
9. The embedding statistics, the chain strength actually used (from `embedding_context`) and $\max|W_{ij}|$ are added.

`skgame/optim/backends.py`, lines 224–295: statistics of a sample set, including the broken chains of every read

```
def sampleset_statistics(ss, P: PlannerProblem, wall: float, polish: bool = False, broken=None,
                         phys_counts=None, discard: bool = False) -> dict:
    """Distinct rewards and their read counts (best first), best spins, QPU timing, chain breaks.

    Rewards are recomputed from the spins, so they are exact in units of R_N whatever scaling
    the sampler applied. With ‘polish‘, every distinct read is also relaxed by greedy single-flip
    ascent (a classical post-processing step that repairs the residual errors of chain breaks
    and of analogue noise) and reported separately.

    With ‘broken‘ (physical rows x N booleans from dwave.embedding.broken_chains) the result also
    has, read by read, how many chains broke:
      out["reads"]                 distinct (configuration, number of broken chains) pairs:
                                   reward, polished_reward, n_broken, count
      info["broken_chains_hist"]   {number of broken chains: number of reads}
      info["chain_break_count"]    for each logical spin i, in how many reads its chain broke
      info["discarded_reads"]      reads dropped by chain_break_method="discard"
    """
    agg = ss.aggregate()
    if len(agg) == 0:                       # chain_break_method="discard" dropped every read
        S = np.zeros((0, P.N))
    else:
        cols = [list(agg.variables).index(i) for i in range(P.N)]
        S = np.asarray(agg.record.sample, float)[:, cols]
    counts = np.asarray(agg.record.num_occurrences, int)
    rewards = np.atleast_1d(P.reward(S)) if len(S) else np.zeros(0)
    order = np.argsort(-rewards)
    info = {"wall_seconds": wall, "num_reads": int(counts.sum())}
    timing = ss.info.get("timing")
    if timing:
```

```
        info["timing_us"] = {k: float(v) for k, v in timing.items() if np.isscalar(v)}
    if "chain_break_fraction" in ss.record.dtype.names and len(ss):
        info["chain_break_fraction_mean"] = float(np.average(ss.record.chain_break_fraction,
                                                             weights=ss.record.num_occurrences))
    empty = len(S) == 0
    out = {"rewards": rewards[order].tolist() if not empty else [], "counts": counts[order].tolist(),
           "best_reward": float(rewards[order[0]]) if not empty else -np.inf,
           "best_S": _spin_string(S[order[0]]) if not empty else "", "info": info}
    polish_of = None
    if polish and empty:
        out.update(polished_rewards=[], polished_counts=[], polished_best_reward=-np.inf, polished_best_S="")
        polish_of = {}
    elif polish:
        from .solvers import _greedy
        Sp = np.array([_greedy(P.W, s.copy())[0] for s in S])
        rp = np.atleast_1d(P.reward(Sp))
        op = np.argsort(-rp)
        out.update(polished_rewards=rp[op].tolist(), polished_counts=counts[op].tolist(),
                   polished_best_reward=float(rp[op[0]]), polished_best_S=_spin_string(Sp[op[0]]))
        polish_of = {S[k].tobytes(): float(rp[k]) for k in range(len(S))}
    if broken is not None:
        nb_phys = broken.sum(axis=1)
        info["chain_break_count"] = (broken * phys_counts[:, None]).sum(axis=0).astype(int).tolist()
        hist = {}
        for nb, c in zip(nb_phys, phys_counts):
            hist[int(nb)] = hist.get(int(nb), 0) + int(c)
        info["broken_chains_hist"] = {str(k): v for k, v in sorted(hist.items())}
        kept = nb_phys == 0 if discard else np.ones(len(nb_phys), bool)
        info["discarded_reads"] = int(phys_counts[~kept].sum())
        nb_rows = nb_phys[kept]
        Sr = (np.asarray(ss.record.sample, float)[:, [list(ss.variables).index(i) for i in range(P.N)]]
              if len(ss) else np.zeros((0, P.N)))
        assert len(Sr) == len(nb_rows), "unembedded rows do not match the physical reads"
        table = {}
        for s_row, nb, c in zip(Sr, nb_rows, ss.record.num_occurrences):
            key = (s_row.tobytes(), int(nb))
            table[key] = table.get(key, 0) + int(c)
        keys = list(table)
        Sk = np.array([np.frombuffer(k[0]) for k in keys]).reshape(len(keys), P.N)
        rk = np.atleast_1d(P.reward(Sk)) if keys else np.zeros(0)
        out["reads"] = {"reward": rk.tolist(), "n_broken": [k[1] for k in keys], "count": [table[k] for k in
     keys],
                        "polished_reward": [polish_of[k[0]] for k in keys] if polish_of is not None else None}
    return out
```

The `seed` lesson. Samplers differ in which arguments they accept. The QPU rejects unknown ones *server-side*, so a Python `try/except TypeError` would never catch the error. Both wrappers therefore pass `seed` only to samplers that declare it in `parameters`:

`skgame/optim/solvers.py`, lines 379–388: passing `seed` only where it is accepted

```
        # only samplers that declare a 'seed' parameter get one: the QPU rejects unknown kwargs
        # server side (not with a TypeError), so trying and catching is not enough there
        params = getattr(self._sampler, "parameters", None)
        if params is None or "seed" in params:
            kw.setdefault("seed", int(rng.integers(2**31)))
        try:
            ss = self._sampler.sample(bqm, **kw)
        except TypeError:
            kw.pop("seed", None)
            ss = self._sampler.sample(bqm, **kw)
```

5 What `qpu --check` does (and why it costs nothing)

`python main_simulation.py qpu --out DIR --check` runs, for each QPU back-end of `config.json`:

1. `make_backend` builds an `AnnealBackend`.

2. `be.largest_problem()` builds `DWaveSampler(solver={"topology__type": ...})`. This contacts SAPI to choose an online solver of that family and download its properties: **metadata only, no problem is submitted**. The method then reads the working graph and calls `busgraph_cache(G).largest_clique()`.

3. The list of calls is built. N above the largest clique is skipped; already-saved results are skipped (resume).

4. `_estimate_qpu_time` computes, for each N, the clique embedding on the real graph and the number of qubits it uses, then evaluates `solver.estimate_qpu_access_time`: **a local formula, no QPU**.

`main_simulation.py`, lines 445–458: the `--check` estimate

```
445 def _estimate_qpu_time(be, todo, log) -> None:
446     """QPU access time still to spend, from the solver's own timing model (no QPU time used)."""
447     if be.device != "qpu":
448         log("[anneal] not a QPU back-end: nothing to estimate")
449         return
450     per_N = {}
451     for N in sorted({u[3] for u in todo}):
452         per_N[N] = be.estimate_access_time(N)
453         if per_N[N] is None:
454             log("[anneal] the solver exposes no timing model")
455             return
456         log(f"[anneal]   N={N:4d}: ~{per_N[N] * 1e3:.0f} ms per call")
457     total = sum(per_N[u[3]] for u in todo)
458     log(f"[anneal] estimated QPU access time for the {len(todo)} remaining calls: {total:.1f} s")
```

`skgame/optim/backends.py`, lines 201–208: one call's estimate from the solver's timing model

```
201     def estimate_access_time(self, N: int) -> float | None:
202         """Seconds of QPU access time for one call, from the solver's timing model (no QPU use)."""
203         solver = getattr(self.child, "solver", None)
204         est = getattr(solver, "estimate_qpu_access_time", None)
205         if est is None:
206             return None
207         n_qubits = chain_stats(self.embedding(N))["n_qubits"]
208         return est(n_qubits, num_reads=self.settings.num_reads, annealing_time=self.settings.annealing_time) / 1
     e6
```

back-end	solver	largest N	calls (`full`)	estimated time
`qpu:zephyr:clique`	`Advantage2_system1`	104	260 ($N \leq 104$)	52.3 s
`qpu:pegasus:clique`	`Advantage_system4`	177	400	105.0 s

Your check (27 October). On the real Zephyr chip the largest all-to-all problem is $N = 104$, far below the 184 of the ideal graph. Defects remove intact line segments (`explanation_project.pdf`, embedding step 10).

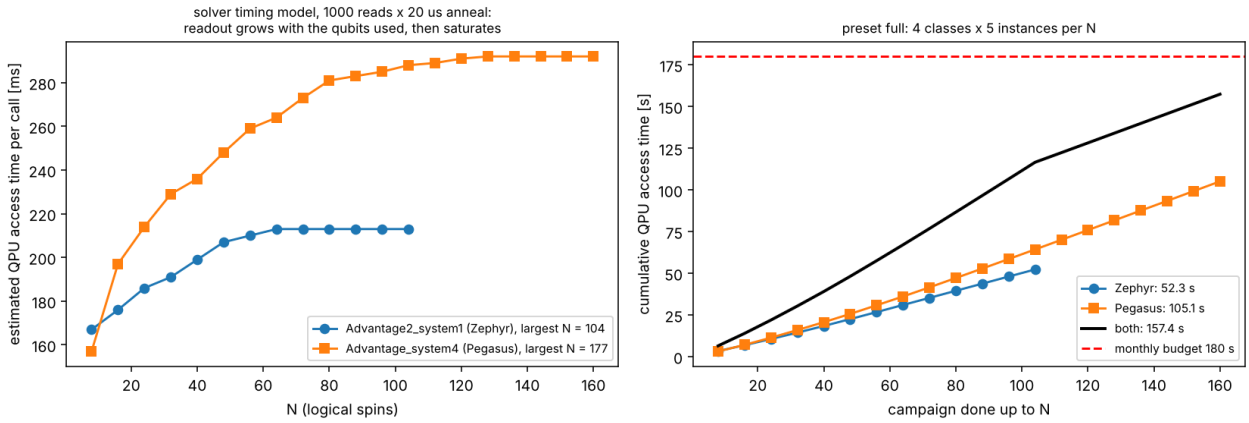


Figure 2: Left: estimated access time per call, from your `--check`. Right: cumulative budget of the `full` campaign: 52.3 s (Zephyr) + 105.0 s (Pegasus) = 157.3 s, against 180 s.

6 How QPU time is estimated: three levels

6.1 Level 1: the rough plan (`main_simulation.py plan`)

For a first idea, before any connection:

$$t_{\text{call}} \approx t_{\text{prog}} + n_{\text{reads}}\,(t_a + t_{\text{ro}} + t_d) = 15\,\text{ms} + 1000\,(20 + 100 + 20)\,\mu\text{s} = 155\,\text{ms}. \tag{1}$$

The constants are generic, and your check shows they **underestimate**: the real per-call times are 157–292 ms. Use level 1 only to compare designs, never to spend.

6.2 Level 2: the solver's timing model (`--check`, budget guard)

D-Wave publishes, in each solver's `properties["problem_timing_data"]`, the parameters of a model that `estimate_qpu_access_time` evaluates. Reading its source (`dwave-cloud-client` 0.14.8) gives

$$t_{\text{access}} = \underbrace{t^{\text{typ}}_{\text{prog}} + t^{\text{therm}}_{\text{prog}}}_{\text{programming}} + n_{\text{reads}}\Big(t_a + t_{\text{ro}}(n_{\text{qubits}}) + t_{\text{delay}} + \max(t^{\text{therm}}_{\text{ro}}, t_{\text{decorr}})\Big), \tag{2}$$

$$t_{\text{ro}}(n_{\text{qubits}}) = \text{piecewise-linear interpolation (often in log–log) of a measured curve.} \tag{3}$$

Two consequences, both visible in your check:

- the cost grows with N, because more qubits mean a longer readout, and then **saturates**;
- Pegasus costs more per call than Zephyr (292 vs 213 ms at large N), because its chains use more qubits.

Above all, the time is **proportional to `num_reads`** apart from the programming term. Halving the reads roughly halves the budget.

6.3 Level 3: the charge (`qpu_access_time`)

After each call the QPU returns its timing. `qpu_access_time` is what Leap charges. It is saved in every result file together with the level-2 estimate (`estimated_access_s`), and `qpu --spent` sums them and prints the ratio actual/estimate:

`main_simulation.py`, lines 427–442: `qpu --spent`: what was actually charged

```
427 def qpu_spent(out: Path, log=print) -> dict:
428     """QPU access time already spent in a campaign, per back-end, from the saved results
```

```
    (the value D-Wave charged for each call), with the ratio to the prediction."""
    rows = {}
    for f in sorted((out / "anneal").glob("qpu_*/*/*.json")) if (out / "anneal").exists() else []:
        info = json.loads(f.read_text())["info"]
        r = rows.setdefault(f.parent.parent.name, {"calls": 0, "spent_s": 0.0, "est_s": 0.0})
        r["calls"] += 1
        r["spent_s"] += info.get("timing_us", {}).get("qpu_access_time", 0.0) / 1e6
        r["est_s"] += info.get("estimated_access_s", 0.0)
    total = sum(r["spent_s"] for r in rows.values())
    for name, r in rows.items():
        ratio = f"{r['spent_s'] / r['est_s']:.2f}" if r["est_s"] else "n/a"
        log(f"[spent] {name:22s} {r['calls']:4d} calls  {r['spent_s']:8.3f} s  (actual/estimate = {ratio})")
    log(f"[spent] total QPU access time in {out}: {total:.3f} s")
    return rows
```

7 The budget guard: `qpu --budget-seconds S`

`main_simulation.py`, lines 331–362: budget object and per-call estimate

```
class QPUBudget:
    """Hard cap on the QPU access time spent by one command (all back-ends together).

    Before each call the cost is predicted with the solver's own timing model
    (`estimate_qpu_access_time`) and inflated by a safety factor; the call is made only if
    spent + inflated estimate <= limit. The factor starts at `safety` and grows if real
    charges come out larger than predicted (factor >= 1.1 x worst observed actual/estimate).
    After each call the REAL `qpu_access_time` returned by the QPU is added to `spent`.
    """

    def __init__(self, limit_s: float | None, safety: float = 1.5):
        self.limit, self.safety, self.spent, self.worst_ratio = limit_s, safety, 0.0, 0.0

    def factor(self) -> float:
        return max(self.safety, 1.1 * self.worst_ratio)

    def allows(self, estimate_s: float) -> bool:
        return self.limit is None or self.spent + self.factor() * estimate_s <= self.limit

    def charge(self, actual_s: float, estimate_s: float) -> None:
        self.spent += actual_s
        if estimate_s > 0 and actual_s > 0:
            self.worst_ratio = max(self.worst_ratio, actual_s / estimate_s)


def _call_estimate(be, N: int, cfg: CampaignConfig) -> float:
    """Predicted QPU access time of one call [s]. Without a timing model (mock QPU) a
    deliberately pessimistic formula: 20 ms programming + reads x (anneal + 300 us)."""
    est = be.estimate_access_time(N) if be.device == "qpu" else 0.0
    if est is None:
        est = 0.020 + cfg.num_reads * (cfg.annealing_time * 1e-6 + 300e-6)
    return float(est)
```

`main_simulation.py`, lines 377–424: the anneal stage: guard before every call, real charge after it

```
def stage_anneal(out: Path, cfg: CampaignConfig, spec: str = "sim:logical", keys=None, max_calls=None,
                 solver_name=None, check_only=False, log=print, retries=3, backend=None,
                 budget: QPUBudget | None = None) -> None:
    be = backend or make_backend(out, cfg, spec, solver_name)
    limit = be.largest_problem()
    log(f"[anneal] backend={be.spec} solver={be.solver_id()} largest_problem={limit}")
    units = list(anneal_units(out, cfg, keys))
    too_big = sorted({u[3] for u in units if limit is not None and u[3] > limit})
    if too_big:
        log(f"[anneal] skipping N={too_big}: above the largest embeddable clique ({limit})")
    todo = [u for u in units if (limit is None or u[3] <= limit)
            and not anneal_path(out, be.label, u[0], u[3], u[4]).exists()]
    log(f"[anneal] {len(units)} calls in the campaign, {len(todo)} still to do")
    if check_only:
        _estimate_qpu_time(be, todo, log)
        return
    total_access = 0.0
    budget = budget or QPUBudget(None)
```

```
    for n, (key, eps, J0, N, b) in enumerate(todo):
        if max_calls is not None and n >= max_calls:
            log(f"[anneal] stopped after --max-calls {max_calls}; rerun to continue")
            break
        est = _call_estimate(be, N, cfg)
        if not budget.allows(est):
            log(f"[anneal] budget: spent {budget.spent:.3f}s of {budget.limit:.3f}s; next call (N={N}) estimated
 "
                f"{est:.3f}s x safety {budget.factor():.2f} would exceed it -> stopped (rerun to continue)")
            break
        J = ensure_instances(out, cfg, key, eps, J0, N)
        P = PlannerProblem(J[b])
        for attempt in range(retries + 1):
            try:
                res = be.sample(P, rng=np.random.SeedSequence([cfg.seed, 3, N, b]))
                break
            except Exception as exc:                    # network hiccups, solver busy, ...
                if attempt == retries or not _transient(exc):
                    raise
                log(f"[anneal] {key} N={N} i={b}: {type(exc).__name__}: {exc}; retry in {2 ** (attempt + 1)}s")
                time.sleep(2 ** (attempt + 1))
        res.update(key=key, eps=eps, J0=J0, N=N, instance=b, backend=be.spec)
        res["info"]["estimated_access_s"] = est
        _write_json(anneal_path(out, be.label, key, N, b), res)
        acc = res["info"].get("timing_us", {}).get("qpu_access_time", 0.0) / 1e6
        budget.charge(acc if acc > 0 else est, est)
        total_access += acc
        cbf = res["info"].get("chain_break_fraction_mean")
        log(f"[anneal] {n + 1:4d}/{len(todo)} {be.label} {key:10s} N={N:4d} i={b} best={res['best_reward']:.5f}
 "
            f"chain_breaks={'-' if cbf is None else f'{cbf:.3f}'} qpu_access={acc * 1e3:.1f}ms (est {est * 1e3
:.0f}ms) "
            f"(total {total_access:.3f}s)")
```

Rule: before each call, with $\hat{t}$ the level-2 estimate,

$$\text{call allowed} \iff t_{\text{spent}} + f\,\hat{t} \le S, \qquad f = \max\left(1.5,\ 1.1 \cdot \max_{\text{past calls}} t_{\text{actual}}/\hat{t}\right). \tag{4}$$

- The guard starts with a 50% safety margin. If the real charges turn out larger than the model, the margin grows automatically.
- After every call t_{spent} grows by the *real* `qpu_access_time`.
- The budget covers all back-ends of one command (Zephyr, then Pegasus).
- When a call is refused, the command stops cleanly. Everything done so far is saved, and the same command continues later (resume).
- Without a timing model (mock QPU) the estimate is a deliberately pessimistic formula: $20\,\text{ms} + n_{\text{reads}}(t_a + 300\,\mu\text{s})$.

It is tested in `tests/test_campaign.py`: the rule itself, a 0.01 s budget that refuses every call, and a 1 s budget that completes and records estimates.

What the guard cannot guarantee. It caps the time *this code* spends, using D-Wave's own numbers. It cannot stop a call once submitted, and it does not see time spent by other programs or other users of the same Leap project. The Leap dashboard remains the authority on what is left.

8 Broken chains, read by read

What is saved. For every call, `sampleset_statistics` (listing above) stores:

- `reads`: one row per distinct pair (configuration, number of broken chains), with the reward, the reward after greedy polishing, the number of broken chains and the number of reads;
- `info.broken_chains_hist`: number of reads with $0, 1, 2, \ldots$ broken chains;

- `info.chain_break_count`: for each logical spin i, in how many reads its chain broke. With equal-length clique chains this should be roughly uniform; outliers point at faulty qubits or couplers;
- `info.chain_lengths`, `info.chain_strength`, `info.discarded_reads` (only with `chain_break_method=discard`).

A chain of spin i is *broken* in a read when its qubits do not all agree. Its logical value is then decided by the chain-break method (majority vote by default). The fraction D-Wave reports, `chain_break_fraction`, is exactly (broken chains)$/N$, and the code checks this.

Analysis. `python main_simulation.py chains --out DIR` (also run by `analyze`, and usable before the classical stage) writes `summary/chain_breaks.csv` and `chain_breaks.png`. For each back-end, N and number of broken chains k they give the fraction of reads, the success probability, and the mean residual, raw and after polishing. The reference is the exact or classical optimum when available, otherwise the best read seen.

`main_simulation.py`, lines 705–760: solution quality vs number of broken chains

```
705 def chain_break_table(out: Path, cfg: CampaignConfig) -> list[dict]:
706     """Solution quality as a function of the number of broken chains in the read.
707 
708     One row per (back-end, problem class, N, number of broken chains): how many reads, and among
709     them the success probability and the mean residual, raw and after greedy polishing. The
710     reference is the same as in `collect` (exact / classical, improved by any annealer read);
711     if the classical stage has not run yet it is the best read of all annealers ("best seen").
712     """
713     if not (out / "anneal").exists():
714         return []
715     labels = sorted(p.name for p in (out / "anneal").glob("*"))
716     rows = []
717     for key, eps, J0 in _classes(cfg):
718         for N in cfg.Ns:
719             recs = {lab: [json.loads(p.read_text()) if p.exists() else None
720                           for p in (anneal_path(out, lab, key, N, b) for b in range(cfg.n_instances))]
721                     for lab in labels}
722             if not any(r for v in recs.values() for r in v):
723                 continue
724             cp = classical_path(out, key, N)
725             if cp.exists():
726                 c = json.loads(cp.read_text())
727                 ref, src = np.array(c["ref"], float), ("exact" if c["ref_exact"] else "classical")
728             else:
729                 ref, src = np.full(cfg.n_instances, -INF), "best_seen"
730             for v in recs.values():
731                 for b, r in enumerate(v):
732                     if r is not None:
733                         ref[b] = max(ref[b], r["best_reward"], r.get("polished_best_reward", -INF))
734             for lab, v in recs.items():
735                 acc = {}
736                 for b, r in enumerate(v):
737                     if r is None or "reads" not in r:
738                         continue
739                     t = r["reads"]
740                     rw, nb, ct = (np.array(t[k], float) for k in ("reward", "n_broken", "count"))
741                     pr = np.array(t["polished_reward"], float) if t.get("polished_reward") else None
742                     ok = _success(rw, ref[b])
743                     okp = _success(pr, ref[b]) if pr is not None else None
744                     for k in np.unique(nb).astype(int):
745                         m = nb == k
746                         a = acc.setdefault(k, {"reads": 0, "succ": 0.0, "succ_p": 0.0, "res": 0.0, "res_p":
    0.0})
747                         a["reads"] += ct[m].sum()
748                         a["succ"] += ct[m][ok[m]].sum()
749                         a["res"] += np.sum(ct[m] * (ref[b] - rw[m]) / abs(ref[b]))
750                         if pr is not None:
751                             a["succ_p"] += ct[m][okp[m]].sum()
752                             a["res_p"] += np.sum(ct[m] * (ref[b] - pr[m]) / abs(ref[b]))
753                 tot = sum(a["reads"] for a in acc.values())
754                 for k, a in sorted(acc.items()):
```

```
                    rows.append({"backend": lab, "problem": key, "N": N, "n_broken": k, "reads": int(a["reads"])
    ,
                                 "frac_reads": a["reads"] / tot, "p_success": a["succ"] / a["reads"],
                                 "p_success_polished": a["succ_p"] / a["reads"],
                                 "residual_mean": a["res"] / a["reads"], "residual_polished_mean": a["res_p"] /
     a["reads"],
                                 "reference": src})
    return rows
```

Example (classical simulator, deliberately weak chains $p = 0.5$). At $N = 16$ on Pegasus, 70% of the reads have no broken chain and hit the optimum in 88% of cases. Reads with one or more broken chains never do, raw. Greedy polishing brings those with 1–2 broken chains back to 84–96%. The residual grows almost linearly with the number of broken chains. On the real QPU, with the default $p = 1.414$, the test of 27 October had chain-break fractions of 0 and 0.001 at $N \leq 16$.

9 Ground states

`python main_simulation.py ground --out DIR` (also run by `analyze`) writes `summary/ground_states.csv`. For every instance it gives:

- the best configuration found (a string of $+/-$, one per agent), with the sign fixed so that agent 0 is $+$, since S and $-S$ have the same energy;
- its reward R_N and its energy $H = -\frac{N}{2}(R_N - \text{offset})$;
- where it came from, and whether it is **certified** (exact enumeration, $N \leq 24$) or only **best known**;
- for every source (classical reference, each annealer raw and polished) whether that source found it, and its Hamming distance from it.

The classical stage now saves the reference *configuration* (`ref_S`, `ref_source`) and not only its value. Each QPU result file already holds `best_S` and `polished_best_S`.

`main_simulation.py`, lines 832–894: the ground-state report

```
def ground_states(out: Path, cfg: CampaignConfig, log=print) -> list[dict]:
    """The ground state (best configuration) of every instance, from every source available.

    Candidates: the classical reference (exact Gray-code enumeration for N <= n_exact: certified;
    otherwise the best classical solver / long parallel tempering: best known) and, for every
    annealing back-end, its best read and its best polished read. The best candidate is reported
    with its reward R_N, energy H = -N (R_N - offset)/2, configuration (+/- per agent, sign fixed
    so that agent 0 is +, since S and -S are equivalent), where it came from, whether it is
    certified, and for each source whether it found it and how far (Hamming distance) its best
    configuration is. Writes summary/ground_states.csv."""
    labels = sorted(p.name for p in (out / "anneal").glob("*")) if (out / "anneal").exists() else []
    rows = []
    for key, eps, J0 in _classes(cfg):
        for N in cfg.Ns:
            cp = classical_path(out, key, N)
            c = json.loads(cp.read_text()) if cp.exists() else None
            if c is None and not any(anneal_path(out, l, key, N, 0).exists() for l in labels):
                continue
            J = ensure_instances(out, cfg, key, eps, J0, N)
            for b in range(cfg.n_instances):
                P = PlannerProblem(J[b])
                cand = []                                   # (reward, spins, column, source)
                if c is not None and c.get("ref_S"):
                    cand.append((c["ref"][b], c["ref_S"][b], "classical", c["ref_source"][b]))
                for lab in labels:
                    ap = anneal_path(out, lab, key, N, b)
                    if ap.exists():
                        r = json.loads(ap.read_text())
                        if r.get("best_S"):
                            cand.append((r["best_reward"], r["best_S"], lab, lab))
```

```
                    if r.get("polished_best_S"):
                        cand.append((r["polished_best_reward"], r["polished_best_S"], lab + "+polish", lab +
 "+polish"))
            if not cand:
                continue
            top = max(t[0] for t in cand)                # ties (to rounding) go to the first source:
 classical/exact
            best = next(t for t in cand if t[0] >= top - TOL * max(1, abs(top)))
            g = _canonical(best[1])
            S = np.array([1.0 if ch == "+" else -1.0 for ch in g])
            certified = bool(c is not None and c.get("ref_exact") and c.get("ref_S"))
            row = {"problem": key, "N": N, "instance": b, "reward": float(P.reward(S)),
                   "energy": float(P.energy(S)), "ground_state": g, "found_by": best[3],
                   "certified": certified, "status": "exact optimum" if certified else "best known"}
            for rw, sp, col, _src in cand:
                row[f"{col}:found"] = bool(rw >= top - TOL * max(1, abs(top)))
                row[f"{col}:hamming"] = sum(x != y for x, y in zip(_canonical(sp), g))
            rows.append(row)
    if rows:
        path = out / "summary" / "ground_states.csv"
        path.parent.mkdir(parents=True, exist_ok=True)
        cols = []
        for r in rows:
            cols += [k for k in r if k not in cols]
        with open(path, "w", newline="") as f:
            w = csv.DictWriter(f, fieldnames=cols)
            w.writeheader()
            w.writerows(rows)
        for r in rows:
            log(f"[ground] {r['problem']:10s} N={r['N']:4d} i={r['instance']} R_N={r['reward']:.5f} "
                f"H={r['energy']:9.4f} {r['status']:13s} by {r['found_by']:24s} {r['ground_state'] if r['N'] <=
 40 else r['ground_state'][:40] + '...'}")
        log(f"[ground] -> {path}")
    else:
        log("[ground] no results yet")
    return rows
```

When the QPU's graph changes. A QPU's working graph changes over time as qubits and couplers go offline or come back. On 2 November the Zephyr chip's largest clique was 105, against 104 on 27 October. An embedding cached on an earlier day may then contain a chain that is no longer connected (`DisconnectedChainError`). `EmbeddingCache.get` therefore re-validates every cached embedding against the current graph (`is_valid_embedding`), recomputes it if it no longer fits, and keeps the old file as `N....stale.json`. Errors in the embedding or the problem are no longer retried (`_transient`), and they occur before any QPU time is used.

10 Ising or QUBO? And is the problem trivial?

Ising, which is equivalent to QUBO. The QPU's native form is Ising: spins $S_i = \pm 1$, fields h_i, couplings J_{ij}. The project sends the SK problem in this form, with $h = 0$ and $J_{ij} = -W_{ij}$ on all $N(N-1)/2$ pairs (`to_bqm`). A QUBO is the same model in binary variables $x_i \in \{0, 1\}$. With $S_i = 2x_i - 1$:

$$\begin{aligned} H &= -\sum_{i<j} W_{ij} S_i S_j = -\sum_{i<j} W_{ij}(2x_i - 1)(2x_j - 1) \\ &= \sum_{i<j} (-4W_{ij})\, x_i x_j + \sum_i \Big(2\sum_{j\neq i} W_{ij}\Big) x_i - \sum_{i<j} W_{ij}. \end{aligned} \tag{5}$$

So $Q_{ij} = -4W_{ij}$, $Q_{ii} = 2\sum_{j\neq i} W_{ij}$, and the offset is $-\sum_{i<j} W_{ij}$. The minimiser is the same (via $x = (S+1)/2$), and so is every energy. `PlannerProblem.to_qubo` returns this form. A test checks it against the Ising form on all 2^9 configurations of an instance, and checks that dimod's `change_vartype` turns one into the other. Sending the QUBO instead would change nothing: dimod converts it to the same Ising problem.

skgame/optim/problem.py, lines 52–64: the QUBO form

```
    def to_qubo(self):
        """The same problem as a QUBO in binary variables x_i in {0, 1}, with S_i = 2 x_i - 1.

        H(S) = -sum_{i<j} W_ij S_i S_j
             = -sum_{i<j} W_ij (2x_i - 1)(2x_j - 1)
             = sum_{i<j} (-4 W_ij) x_i x_j + sum_i (2 sum_{j != i} W_ij) x_i - sum_{i<j} W_ij,
        so Q_ij = -4 W_ij (i < j), Q_ii = 2 sum_{j != i} W_ij, offset = -sum_{i<j} W_ij.
        Same minimiser (mapped by x = (S + 1) / 2) and same energy. The QPU works in Ising form;
        dimod converts between the two exactly (bqm.change_vartype)."""
        iu = np.triu_indices(self.N, 1)
        Q = {(int(i), int(j)): -4.0 * float(self.W[i, j]) for i, j in zip(*iu)}
        Q.update({(i, i): 2.0 * float(self.W[i].sum()) for i in range(self.N)})
        return Q, -float(self.W[iu].sum())
```

Is it trivial? The problem is not:

- the SK ground state is a dense spin glass with frustration and exponentially many local minima. Its worst case is NP-hard, and typical instances need exponential time for every known exact method;
- there is no field ($h = 0$), so S and $-S$ are both ground states. The code counts either as success.

What *is* trivial is the 1-second test: $N = 8, 16$ is solved exactly by a laptop in milliseconds. It tests the machinery (token, embedding, timing, budget), not the QPU's ability. The campaign's real question starts at large N, up to 104 on Zephyr and 177 on Pegasus. There, good classical solvers (parallel tempering, tabu) remain very strong on dense SK, and the QPU is handicapped by the all-to-all embedding: long chains, couplings compressed to ~ 0.05, analogue noise. The honest expectation is not that the QPU wins. The campaign measures where it stops finding the optimum, how that depends on broken chains, and whether Zephyr's shorter chains move that point. The `ferro_easy` regime ($J_0 = 3$) is *deliberately* easy: it is the control case where every method should succeed.

11 From reads to results

main_simulation.py, lines 498–528: per back-end metrics in `collect`

```
            for s, recs in ann.items():
                done = [(b, r) for b, r in enumerate(recs) if r is not None]
                if not done:
                    continue
                p = np.array([np.sum(np.array(r["counts"])[_success(r["rewards"], ref[b])]) / max(1, np.sum(r["
    counts"]))
                              for b, r in done])
                best_res = np.array([(ref[b] - r["best_reward"]) / abs(ref[b]) for b, r in done])
                mean_res = np.array([np.average((ref[b] - np.array(r["rewards"])) / abs(ref[b]), weights=r["
    counts"])
                                     if np.sum(r["counts"]) else np.nan for b, r in done])
                t_read = np.median([_time_per_read(r) for b, r in done])
                acc = [r["info"].get("timing_us", {}).get("qpu_access_time", np.nan) / 1e6 for b, r in done]
                cbf = [r["info"].get("chain_break_fraction_mean", np.nan) for b, r in done]
                extra = {}
                if all("polished_rewards" in r for b, r in done):
                    pp = np.array([np.sum(np.array(r["polished_counts"])[_success(r["polished_rewards"], ref[b])
    ])
                                   / max(1, np.sum(r["polished_counts"])) for b, r in done])
                    extra.update(p_success_polished=float(np.median(pp)),
                                 residual_polished=float(np.median([(ref[b] - r["polished_best_reward"]) / abs(
    ref[b])
                                                                    for b, r in done])))
                emb = [r["info"].get("embedding") for b, r in done]
                if all(emb):
                    extra.update(chain_len_mean=float(np.mean([e["chain_len_mean"] for e in emb])),
                                 chain_len_max=int(max(e["chain_len_max"] for e in emb)),
```

```
                                n_qubits=int(np.mean([e["n_qubits"] for e in emb])),
                                chain_strength=_nan_to_none(float(np.mean([r["info"].get("chain_strength") or
    np.nan
                                                                          for b, r in done]))))
            rows.append(dict(base, kind="anneal", method=f"anneal_{s}", regime="", n_done=len(done), **extra
    ,
                                p_success_median=float(np.median(p)), p_success_mean=float(p.mean()),
                                p_best_found=float(np.mean(best_res <= TOL)),
                                residual_median=float(np.median(best_res)), residual_mean_read=float(np.
    nanmedian(mean_res)) if np.isfinite(mean_res).any() else None,
                                seconds_median=t_read, tts99=_tts(p.mean(), t_read),
```

For each problem class, N and instance b, with reference R_{ref} (exact for $N \le 24$; otherwise the best of the classical solvers and of every annealer read, polished reads included):

$$p_{\text{success}} = \frac{\#\{\text{reads with } R \ge R_{\text{ref}} - 10^{-9}\}}{n_{\text{reads}}} \quad \text{(per read)}, \tag{6}$$

$$\text{residual} = \frac{R_{\text{ref}} - R_{\text{best read}}}{|R_{\text{ref}}|}, \tag{7}$$

$$t_{\text{read}} = \frac{t_{\text{sampling}}}{n_{\text{reads}}} \quad (t_{\text{sampling}} = \texttt{qpu_sampling_time}), \tag{8}$$

$$\text{TTS}_{99} = t_{\text{read}} \frac{\ln 0.01}{\ln(1-p)}. \tag{9}$$

`main_simulation.py`, lines 548–563: time per read and TTS_{99}

```
548 def _time_per_read(r) -> float:
549     """QPU: sampling time per read (anneal + readout + delay); simulator: wall time per read."""
550     t = r["info"].get("timing_us", {})
551     n = r["info"]["num_reads"]
552     if "qpu_sampling_time" in t:
553         return t["qpu_sampling_time"] / 1e6 / n
554     return r["info"]["wall_seconds"] / n
555
556
557 def _tts(p: float, t: float) -> float:
558     """Time to reach the reference with 99 % confidence: t ln(0.01) / ln(1 - p)."""
559     if p >= 1:
560         return float(t)
561     if p <= 0:
562         return INF
563     return float(t * math.log(0.01) / math.log1p(-p))
```

`compare_backends` ranks the back-ends of the same device for each (class, N): higher median p_{success} wins, ties go to the lower residual. The output is `summary/backend_comparison.csv` and `.png`.

12 How the D-Wave path was tested

- `test_embedding.py`: clique embeddings are valid on both topologies (every chain connected, chains disjoint, every pair linked); Zephyr chains are shorter at production size; the cache round-trips; back-end specs parse; the embedded simulator finds the exact ground state, and the chain strength equals $p\,J_{\text{rms}}\sqrt{N-1}$ computed independently.
- `test_campaign.py`: every read is kept and `seed` is skipped where unsupported (with `ExactSolver`); the whole campaign runs end to end and resumes; **both QPU back-ends run against** `MockDWaveSampler` with Zephyr and Pegasus topologies, using the real `DWaveSampler` code path from `solver={topology__type}` onwards; the budget guard (rule, refusal, completion, `--spent`) is checked.
- What the mock cannot test: the real timing values, the real noise, SAPI errors and Leap accounting. That is what the one-second test below is for.

13 The one-second test, and a plan for the 180 s

13.1 Test: at most 1 s of QPU time, both topologies

A tiny separate campaign: $N = 8$ and 16, one instance, one problem class, 100 reads per call, both QPUs. That makes 4 calls. At 100 reads each the timing model gives a few tens of ms per call, so the whole test costs a fraction of a second. The guard makes **1 s a hard ceiling** anyway.

```
cd ...\Dimod\src\unlearnable_games_explanation      # (PowerShell, .venv active)
$T = "results\qpu_test"
python main_simulation.py init --preset smoke --out $T --Ns 8,16 --n-instances 1 --regimes
    sg_satisficing --num-reads 100
python main_simulation.py qpu --out $T --check                  # predicted cost: spends
    nothing
python main_simulation.py qpu --out $T --budget-seconds 1       # the test: <= 1 s,
    normally ~0.1-0.2 s
python main_simulation.py qpu --out $T --spent                  # what Leap charged, and
    actual/estimate
python main_simulation.py classical --out $T                    # seconds, for the
    reference optimum
python main_simulation.py analyze --out $T                      # success per read, chain
    breaks
```

What to check in the output:

1. every call prints `qpu_access=...ms (est ...ms)`: real vs predicted;
2. `--spent`: the total, well below 1 s, and `actual/estimate` close to 1;
3. `chain_breaks`: should be 0 or tiny at $N = 8, 16$;
4. the Leap dashboard shows the same usage.

13.2 Spending the 180 s before 4 November

Your check gives 157.3 s for the full campaign on both topologies at 1000 reads. That leaves about 22 s of margin, *if* the real charges match the model, which the test will show. Two safe options:

plan	QPU time	comment
`full`, 1000 reads, both QPUs	≈ 157 s	as designed; run with `--budget-seconds 170` so it can never pass 170 s
`full`, 500 reads (`init ... --num-reads 500`)	≈ 80 s	time $\propto$ reads; leaves ≈ 90 s for a chain-strength scan at the N where success drops

```
$F = "results\full"
python main_simulation.py init --preset full --out $F --force          # (add --num-reads
    500 for option 2)
python main_simulation.py qpu --out $F --check
python main_simulation.py qpu --out $F --budget-seconds 170            # stops by itself
    before 170 s
python main_simulation.py qpu --out $F --spent
```

The QPU stage does not need the classical stage. That can run before or after, on the laptop or on Leonardo, and `analyze` combines them. If the run stops on the budget, it has saved everything done so far, and `--spent` shows the total.

References

- D-Wave, Ocean documentation: https://docs.dwavequantum.com/en/latest/ocean/; Leap authorisation: https://docs.dwavequantum.com/en/latest/ocean/leap_authorization.html.
- D-Wave, QPU timing and `estimate_qpu_access_time`: `dwave.cloud.solver.StructuredSolver` (source read for §6, dwave-cloud-client 0.14.8).
- Boothby, King, Raymond, *Fast clique minor generation in Chimera qubit connectivity graphs*, Quantum Inf. Process. 15, 495 (2016): the clique embedding.
- Cai, Macready, Roy, *A practical heuristic for finding graph minors*, arXiv:1406.2741 (2014): minorminer.
- Cineca, Quantum Computing as a Service: https://www.quantumcomputinglab.cineca.it/en/resources.
- Project: `Dimod/explanation_project.pdf` (SK game, algorithms, embedding, chains, noise), `Dimod/LEAP_TOKEN.md`, `README_LOCAL.md`, `README_CINECA.md`.